Minecraft Multi-Agent Studio 架构评审报告

评审日期：2026-08-08 | 评审对象：`C:\Users\baichengbo\Documents\minecraft-agents`（分支 agent/control-api-coordinator-bridge，提交 40b406e） | 评审依据：ARCHITECTURE.md / ROADMAP.md / CHANGELOG.md + 全部源码结构扫描 | 用途：Codex 接管开发前的基线评审

一页结论：架构方向正确、安全边界严谨（同类项目中少见），主要债务是**约 3700 行重复的链路代码**、**权威状态不持久化**和**三个千行巨型文件**。属于"结构良好、需要一次收敛重构"的状态，**不需要推翻重来**。当前阶段**不建议引入 Rust**（无任何已测量的性能瓶颈，且核心依赖 Mineflayer 锁定 Node 生态），但为寻路计算和接入网关预留了两个未来的 Rust 位置及明确触发条件。MVP 定义为"单 Bot 自主完成一件真实任务、全程可观测可停止、崩溃可恢复"，现有代码已完成约 70%。

一、项目现状快照

pnpm TypeScript monorepo，四个应用 + 六个共享包，共约 1.2 万行业务源码（不含测试）。Node 22、Mineflayer 4.37.1、Next.js 16、PostgreSQL + BullMQ。

层	模块	核心文件（行数）	状态
前端	apps/dashboard	Next.js 页面 + 服务端代理 + Zustand Store	完成 浏览器 E2E 通过
API 层	apps/control-api	server.ts (912) / coordinator-link-client.ts (1036)	完成 四进程验收通过
控制平面	apps/coordinator	control-link-server.ts (1068) / runtime-link-server.ts (790) / bot-registry.ts (416) / stop-controller.ts (455)	完成 但状态全在内存
执行层	apps/bot-runtime	runtime.ts (971) / runtime-link-client.ts (853) / mineflayer-client.ts (565)	完成 真实联机验收通过（1.21.8）
协议	packages/protocol	bot-messages.ts (604) 等，全部 Zod v1 Schema	完成 项目最有价值资产
安全	packages/safety	index.ts (248)，确定性 L0 反射	离线完成 实机危险矩阵未做
技能	packages/skills	executor.ts (416)，仅 movement.goto@1.0.0	离线完成 实机寻路未验收
任务	packages/task-system	postgres-repository.ts (1420) / bullmq-worker.ts (1125) / outbox-reconciler.ts (226, WIP)	进行中 P1.3.2 起步代码已提交

验收基线（截至 2026-08-07）：224 项离线测试通过、生产构建通过、四进程服务级验收两名执行者独立通过、GitHub Actions 真实 PostgreSQL/Redis 集成门通过（运行 30315095898）。

二、整体架构合理性分析

2.1 应保留的设计基因（不许任何重构破坏）

- **协议先行 + fail-closed**：所有跨进程消息在 packages/protocol 用 Zod 严格校验，显式 schemaVersion，未知字段/版本默认拒绝；链路断开时返回 503 并清空镜像，绝不沿用旧状态或伪造成功。
- **控制平面分层清晰**：Browser → Dashboard → Control API → Coordinator → Runtime 五层职责单一，权限逐层收窄；浏览器拿不到长期令牌，坐标读取需单独权限，停止命令需二次确认 + 本地确认码 + Runtime 真实回执。
- **确定性优先的 AI 边界**：L0 安全层和技能执行器不接受模型文本；未来模型只能输出白名单技能调用 JSON，且经 Schema、Policy、参数边界三重验证。这为 P2 智能层预留了安全接口。
- **诚实的验收文化**：文档明确区分"已测试/兼容未测试/不支持"、"离线验收/实机验收"，不冒充完成。

2.2 存在的问题（按严重度排序）

#	问题	证据	影响
1	**同一个"认证 WS 链路"模式手写了两遍**	runtime-link-server(790) + runtime-link-client(853) 与 control-link-server(1068) + coordinator-link-client(1036)，共约 3700 行；HMAC/timingSafeEqual 逻辑散布在 6 个文件	修一个握手 bug 要改四处；是 AI 接管后最容易改错的地方
2	**权威状态全在内存**	Bot 注册表、停止幂等缓存、审计记录随 Coordinator 重启蒸发	审计不落盘与项目安全叙事自相矛盾；重启后 Bot 必须重新上报
3	**三个千行巨型文件**	postgres-repository.ts (1420)、bullmq-worker.ts (1125)、control-link-server.ts (1068)	AI 在此类文件上做局部修改的出错率显著上升
4	任务系统与技能系统未接通	Coordinator 尚无任务调度入口；TaskDispatchV1 → SkillInvocationV1 的链路缺失	两套已验收的系统各自孤立，无法产生业务价值
5	多 Bot 无进程编排概念	SingleBotRuntime 一进程一 Bot（设计正确），但无 supervisor	10 个 Bot = 10 次手动启动
6	验收碎片化	双进程/四进程/浏览器 E2E/真实联机各自独立	始终没有一条"全真"端到端路径
7	开发体验摩擦	启动四进程需手工跑多段 PowerShell 生成密钥/哈希并跨窗口复制	诱使使用者走捷径存明文密钥，反而降低安全

三、目标系统架构图（最终态）



与现状的差异：① 新增 `packages/secure-link` 共享库，合并两条链路的重复实现；② Coordinator 增加任务调度器、多 Bot 进程编排、PG 持久化审计；③ 新增独立任务 Worker 应用（apps/task-worker）；④ AI Router 作为旁路而非链路中的一环——它宕机不影响手动控制。

四、Rust vs TypeScript 决策

4.1 结论：当前阶段全部保持 TypeScript

这个系统 95% 是 **I/O 密集的控制平面**（WebSocket 转发、JSON 校验、数据库读写）。Rust 在这些场景的收益接近零，而成本很实际：

- **Bot Runtime 被 Mineflayer 锁死在 Node。**27 个 Minecraft 版本的协议适配全靠它；Rust 对应物 azalea 在成熟度、版本覆盖、插件生态上差距很大，迁移等于重写并作废所有已通过的联机验收。
- 每秒几十条心跳/状态消息的吞吐对 Node 毫无压力，**当前没有任何一个已测量的性能瓶颈。**
- Zod Schema 是全链路的单一事实源，TS 类型自动推导；引入第二语言即产生协议双重维护。
- 对 AI 接管而言，单语言栈的修改出错率远低于混合栈。

4.2 预留的两个 Rust 位置（含触发条件）

模块	触发条件（必须先测量）	接口边界	理由
Rust 寻路/世界几何服务	多 Bot 后寻路计算开始阻塞 Runtime 事件循环（事件循环延迟 > 50ms 可测量）	独立进程，结构化 JSON/gRPC，作为一个"能力适配器"接入	A*/体素几何是纯 CPU 计算，是 Rust 的甜点区；不碰 Mineflayer
Rust 接入网关	Bot 数量 50+ 且 Coordinator 单进程成为已测量瓶颈	无状态 WS 终结/转发，协议已版本化可独立替换	高并发连接管理，无业务规则

原则：Rust 只放在有清晰 JSON 协议边界、纯计算或纯转发、可独立验收的**叶子位置**，绝不进入含业务规则的控制平面。在有性能测量数据之前写 Rust 是徒增复杂度。

4.3 保持 TypeScript 的模块清单

模块	语言	理由
packages/protocol	TS	Zod 是校验基石，全链路共享类型推导，换语言即失去单一事实源
apps/bot-runtime	TS	Mineflayer 硬依赖，不可移
apps/coordinator / control-api	TS	I/O + 业务规则密集，与 protocol 同语言收益最大
apps/dashboard	TS	Next.js 生态
packages/task-system	TS	pg/BullMQ 都是 Node 原生生态，逻辑复杂但不吃 CPU
packages/safety / skills / observability / environment	TS	safety 每 tick 只做阈值比较（纳秒级），无性能议题

五、第一版 MVP 设计

5.1 MVP 定义

> **一个 Bot 在隔离测试服自主完成一件真实有用的事，全程可观测、可停止、崩溃可恢复。**
> 现有代码已完成约 70%——MVP 是补全闭环，不是新起炉灶。

5.2 范围内工作（按依赖顺序）

#	工作项	说明	验收标准
1	P1.3.2 持久任务闭环	事务 outbox + reconciler（已有 WIP 提交 40b406e）、独立 Worker 应用、跨 Worker 取消、自动过期租约恢复	杀掉任意进程后任务不丢、不重、可恢复
2	Coordinator 任务调度接入	Coordinator 从 PG 领取任务 → TaskDispatchV1 → Worker → SkillInvocationV1 → Runtime。**这是目前完全缺失的一环**	Dashboard 创建的任务能驱动 Bot 真实动作
3	审计与注册表快照落 PG	复用 task-system 连接层，append-only 审计表	Coordinator 重启后审计不丢
4	2~3 个生产技能	gather.wood / craft.item 级别，走既有技能框架与能力适配器	每个技能独立离线 + 实机场景测试
5	L0 实机危险场景矩阵	在真实隔离服构造溺水/岩浆/坠落/饥饿场景	L0 真实动作全部正确触发——自主执行任务前的安全前提
6	Dashboard 任务看板	创建任务、看状态流转、取消任务	浏览器 E2E 覆盖

5.3 明确砍掉的范围

多 Bot 协调、AI/模型接入、Docker/便携/网吧模式、正式服务器、Rust、长期记忆、跨主机 TLS。

5.4 MVP 端到端验收场景

```
Dashboard 创建"采集 16 个原木"任务
  → Coordinator 从 PG 领取并派发（outbox 保证原子）
  → Worker 领取 → SkillInvocationV1 → Bot 在隔离服真实执行
  → 期间人为制造一次溺水，L0 正确救回并恢复任务
  → 中途杀掉 Worker 进程，租约过期后任务被自动回收重派
  → 完成后 Dashboard 显示结果与产物数量
  → 全程同一 traceId 贯穿五个进程的结构化日志，审计可查
```

六、未来扩展路线

阶段	内容	说明
v1.1 收敛重构（MVP 后立即做）	抽 packages/secure-link 消灭 3700 行重复；拆三个千行文件；加 .gitattributes 固定行尾；一键 dev-up 脚本（自动生成密钥并拉起四进程）	为 AI 长期接管降错误率，优先级高于新功能；趁规模小时做成本最低
v2 多 Bot（对应 P1.4~P1.6）	Coordinator 进程编排（supervisor 拉起/回收 Runtime）、资源/区域/箱子锁、角色分配、Prometheus + 集中日志、生产技能扩到 10+	编排是新的复杂度台阶，需要 v1.1 的干净底座
v3 智能层（对应 P2.1~P2.3）	AI Router + 本地模型规划（输出只能是技能调用 JSON）、pgvector 长期记忆、云专家失败升级	安全边界不变：模型永远不碰 tick、不出白名单
v4 规模与硬化	按测量数据决定是否引入 Rust 寻路/网关；跨主机部署（TLS + 密钥轮换）；受监管自我改进、基地撤离、战斗模块	战斗与自我改进保持最后接入，与既有 ROADMAP 一致

七、给 Codex 接管的操作建议

1. **更新 AGENTS.md**：把本报告的模块边界、"Rust 触发条件"和"不许破坏的设计基因"（第 2.1 节）写进去，防止 AI 自作主张重写已验收模块。
2. **顺序建议**：如果追求稳，先做 v1.1 的 secure-link 抽取再做 MVP；如果急于见效，MVP 第 1、2 项可以先行，但 v1.1 必须排在 v2 多 Bot 之前。
3. **第一天清理**：删掉 runtime-data 里的 e2e 日志和两份 PR 草稿（pr-1-body.md、pull-request-runtime-link.md）；加 .gitattributes。
4. **保持验收文化**：每个模块必须先过 pnpm check（类型 + 224 项测试 + 构建）才算完成；新功能必须附带测试；不冒充"已验证"。
5. **安全红线不动**：只连隔离测试服、不连正式服务器、密钥不落盘不进日志、模型输出不直接执行。

本报告由 Claude（Fable 5）基于 2026-08-08 的代码库状态生成。数据来源：ARCHITECTURE.md、ROADMAP.md、CHANGELOG.md、README_CN.md、PROJECT_AUDIT.md 及 apps/、packages/ 全部源码的结构扫描（文件行数、跨包依赖、HMAC 实现分布）。验收数字（224 项测试、四进程验收）来自 2026-08-08 本机实际运行的 pnpm check。